WGIMS

Complete System Documentation & QA Guide

Welfare Goods Inventory Management System

A beginner-friendly explanation of how the system works, written for QA testers and evaluators.

Version: 1.0 (based on the current source code)
Prepared: August 14, 2026
Application: WGIMSv2 — Laravel 12 / PHP 8.2 / MySQL

Table of Contents

1. System Overview

WGIMS (Welfare Goods Inventory Management System) is a web-based inventory system used by a provincial/regional welfare office to manage welfare goods distributed to disaster victims and similar beneficiaries. It tracks the entire life of every item:

1. **Receipt** — Goods arrive from a supplier under a *Delivery / Subsidy* document (with a DR number).
2. **Storage** — Goods are stored in *Warehouses* (called centers), each with its own stock records and stock numbers.
3. **Issuance** — Goods are released to requesting units through *RIS* (Requisition and Issue Slip) documents.
4. **Movement** — Goods can be transferred between warehouses through *Stock Transfers*.
5. **Reporting** — The system produces government-standard reports: *RPCI* (Report on Physical Count of Inventories), *RSMI* (Report of Supplies and Materials Issued), *Inventory Balance*, and per-item *Stock Cards*.

The system is **multi-warehouse** and **role-based**: five different user roles see and do different things. It keeps a full audit trail of corrections, and it protects financial accuracy by preventing destructive deletes of records that are already in use.

> **Plain-language summary:** Think of WGIMS as the digital store room ledger. Every time goods come in, go out, or move between rooms, WGIMS updates the ledger automatically and generates the official government forms that the office must submit.

Who uses it

- **Administrator** — full control, all warehouses, creates users, manages settings.
- **Warehouse Manager** — sees everything (read-only on most edits), can create new records and approve requests.
- **Supply Custodian** — manages a specific warehouse; can approve requests for their warehouse.
- **Warehouse Head** — manages a specific warehouse; can approve requests.
- **Warehouse Staff** — can view their warehouse's data and record deliveries, but cannot create new documents on their own.

2. Technology Stack

Component	What is used	Why it matters to QA
Server language	PHP 8.2 (Laravel 12 framework)	All business logic runs on the server; the browser is just the interface.
Web server	XAMPP / Apache (development), Laravel serving the app	App is reached through a web browser at the local server address.
Database	MySQL (database name: `wgims`)	All items, documents, users, and logs are stored here.
Frontend	Blade templates, Tailwind CSS 4, JavaScript (via Vite build tool, Axios for AJAX)	Pages are server-rendered; some actions (dropdowns, checks, notifications) use AJAX.
Excel export	PhpSpreadsheet library	RPCI / RSMI / Inventory Balance can be exported to .xlsx files.
Authentication	Custom login (username + password, hashed), session-based	Users log in with a username; passwords are hashed in the database.
Tests	PHPUnit (in-memory SQLite database)	The codebase includes automated tests that QA can run.

Config values live in the `.env` file. The app name is `WGIMSv2`. Debug mode is currently enabled (`APP_DEBUG=true`), which shows detailed error pages during development.

> 🟥 **Note for production:** `APP_DEBUG=true` is a development setting. In a live deployment this should be set to `false` so visitors cannot see internal error details. This is expected during development/QA.

3. Modules (Screen-by-Screen)

The system is organized into these modules. Each row lists what a typical user sees and what the screen does.

3.1 Dashboard

Screen	What it shows / does
Admin Dashboard (`/`)	Account balances per warehouse per category (quantity + peso value), grand totals, unliquidated subsidies (still pending/partial), and summary stats (total items, subsidies, pending RIS, warehouses).
Warehouse Dashboard	Same style of balance summary but only for the warehouses assigned to the logged-in user. If the user has no warehouse, a friendly "no warehouse" message is shown.

3.2 Items & Categories

Screen	What it shows / does
Items list (`/items`)	All stock items with warehouse, stock number, category, unit, quantity, unit cost, ENGAS unit cost, reorder point, expiry. Supports search (description/stock no./RIS no.) and filters (category, warehouse, stock status in/out/low, source subsidy deleted/archived/active). Admins see inactive/out-of-stock items too.
Add Item	Creates an item in a chosen warehouse. A stock number is assigned later when a delivery is recorded (not at creation).
Item detail	Shows the item and its stock card entries.
Edit Item	Admin can edit details including ENGAS unit cost; only the full admin can change ENGAS cost.
Item Categories (`/item-categories`)	Admin-only. Manages the category list (e.g., FOOD, NON-FOOD) with account codes and sort order. Each category can hold a "catalog" of pre-defined item names used by the delivery form.

3.3 Delivery / Subsidy

Screen	What it shows / does

List (`/delivery-subsidies`)	All delivery/subsidy documents with supplier, DR no., date, quantity requested, quantity delivered so far, status. Filterable by status, description, account code, warehouse.
New Delivery/Subsidy	Admin/WM create a document: RIS no. (becomes the DR no.), supplier, date, and line items chosen from the item catalog. Quantity, unit, category, expiry per line. Unit cost and warehouse are NOT set yet — they are decided when the delivery is recorded.
Detail / Record Delivery	Records the actual receipt: which warehouse receives each line, unit cost, ENGAS unit cost, quantities, condition, expiry, received-by. This is where stock quantities increase and stock cards are created.
Edit / Correct	Admin-only. Edit header and lines (lines that already have deliveries are locked). The "Correct" screen allows fixing the document; corrections cascade to dependent records with an audit log.
Archive / Restore	Admin-only. Soft-"freezes" a subsidy without deleting it; archived subsidies still show their trail on items/transfers.
Audit Log	Admin-only. History of every update/correction with who, what, before/after.

3.4 Requisitions (RIS)

Screen	What it shows / does
List (`/requisitions`)	All RIS documents with status (pending/approved/partially_approved/cancelled), date, requesting office/LGU.
Create RIS	Anyone with a warehouse can create: header info (entity, fund cluster, office, division, responsibility center code, requesting LGU, purpose, dates) plus line items with quantities. Items are description-level at this stage.
Approve (`/requisitions/{id}/approve`)	User with approve right picks which stock records fulfill each line and how much is issued (dispatch). Partial approval is supported. This decreases stock, creates stock card issuance entries, and links to the exact stock.
Dispatch edit	Admin can edit an individual dispatch (which stock, quantity, cost, expiry, DR no.) after approval; corrections are tracked.
Signatories / Print	Prints the official RIS form with signatory names.

Correction / Audit Log	Admin can correct an approved RIS (with rules that protect already-issued quantities) and view the correction history.

3.5 Stock Transfers

Screen	What it shows / does
List (`/transfers`)	All transfers between warehouses, searchable by transfer no., source RIS no., source DR no., and filterable by source subsidy status.
Create Transfer	Admin/WM pick source warehouse, destination warehouse, date, and items + quantities to move.
Dispatch Transfer	Confirms the move: source stock decreases, destination stock increases (or is created), stock cards record both sides, and the transfer keeps a snapshot of the original subsidy (RIS/DR) for tracing.
Print	Prints the transfer document.

3.6 Stock Cards

Screen	What it shows / does
Stock Cards Summary (`/stock-cards`, `/stock-cards/summary`)	Summary of all items grouped by category, showing quantity, unit cost, ENGAS unit cost, and total value.
Stock Cards per Category (`/stock-cards/{category}`)	List of items in a category with their running balances.
Item History (`/stock-cards/item/{id}/history`)	The full stock card: every receipt/issuance with date, reference, in/out, balance after each move. Admin additionally sees an "ENGAS Value" card when ENGAS cost is set.
History by Unit Cost (FIFO view)	Shows the same history broken into cost "batches" (FIFO layering) so evaluators can verify cost valuation.

Print Stock Card (`/stock-cards/item/{id}/print`)	Prints the standard stock card form (unit cost only — no ENGAS columns by design).

3.7 Reports

Screen	What it shows / does
RPCI (`/reports/rpci`)	Report on the Physical Count of Inventories: stock number, description, unit, unit cost, ENGAS unit cost, qty per card, qty per count, total value, remarks. Printable and exportable to Excel.
RSMI (`/reports/rsmi`)	Report of Supplies and Materials Issued: built from approved/partially-approved requisitions with issued quantities, grouped by RIS, with subtotals, grand total, and stock-number recapitulation. Printable and exportable.
Inventory Balance (`/reports/inventory-balance`)	Admin-only. Per-warehouse, per-category, per-description listing of all positive-quantity stock with quantities, unit cost, ENGAS cost, total value, and grand totals. Exportable to Excel.
Snapshots	RPCI/RSMI can be saved as monthly snapshots (period-month) so past reports can be re-viewed later.

3.8 Reference Data & Administration

Screen	What it shows / does
Suppliers (`/suppliers`)	Supplier master list (name, address, TIN, contact person, phone, email) with activate/deactivate.
Warehouses (`/warehouses`)	Warehouse master list (name, code, place, active flag) with user counts. Users can see only their assigned warehouses; admins see all.
Users (`/users`)	Admin-only. Create/edit users, assign role and warehouses, enable/disable accounts. Admin and Warehouse Manager roles have no warehouse assignment (they see all); all other roles must be assigned at least one warehouse.
Notifications (bell icon)	Per-user notifications (e.g., "new delivery/subsidy created") with read/unread state and a dropdown badge.

4. Data Flow (Big Picture)

Below is how data moves through the system, in the exact order the office would use it. Every arrow below is backed by a database record, so any quantity on any report can be traced back to its source document.

1. **Setup** — An admin creates Warehouses, Suppliers, Item Categories, and Users (with roles + warehouse assignments).
2. **Receive (Delivery/Subsidy)**
    - Admin/Warehouse Manager creates a Delivery/Subsidy document (DR number = RIS number typed, with a suffix if the DR already exists).
    - The receiving staff records the actual delivery: for each line, choose the warehouse, unit cost, ENGAS unit cost, quantity delivered, and condition.
    - The system finds (or creates) the exact `items` record (matching warehouse + description + unit + category + unit cost, and optionally expiry). A unique **stock number** is generated if the item doesn't have one yet.
    - Item quantity increases. A **stock card entry** (receipt) is created. The item is tagged with the source subsidy (RIS/DR snapshot) so it can be traced later.
    - The subsidy's `qty_delivered` and status (pending/partial/full) update automatically.
3. **Issue (Requisition / RIS)**
    - A user creates an RIS with line items (description-level).
    - An approver (admin, warehouse manager, custodian, or head) approves: for each line, they pick the actual stock record and quantity to issue. This is the *dispatch*.
    - Each dispatch: decreases that item's quantity, creates a stock card issuance entry (linked to the exact dispatch record), and optionally sets a DR number and expiry for the issued stock.
    - The RIS status becomes `approved` or `partially_approved` (if only some lines/quantities were issued).
4. **Move (Stock Transfer)**
    - Admin/WM creates a transfer: source warehouse → destination warehouse, items + quantities.
    - On dispatch: source item decreases, destination item increases (found or created by the same matching rules), two stock card entries are written (issuance on source, receipt on destination), and the transfer snapshots the source subsidy's RIS/DR.
5. **Report**
    - RPCI reads live item balances. RSMI reads issued quantities from approved RIS documents. Inventory Balance reads live item balances for admin. All can be printed or exported to Excel.

> **Key idea for QA:** The system is *document-driven*. Quantities are never typed directly into the stock; they always come from a delivery, a requisition dispatch, or a transfer. If a number looks wrong, the fix is made on the document (correction), never by editing the stock balance by hand.

5. Delivery / Subsidy Logic (in detail)

5.1 Creating a Delivery/Subsidy

- Allowed roles: Admin, Warehouse Manager (route middleware `admin.create`). Center Staff cannot create.
- Validation on create: `ris_number` (required), `supplier_id` (must exist), `date` (required date), and at least one line item. Each line needs: description, unit, a valid category, quantity ≥ 0.01, optional expiry, optional catalog item reference, optional account code.
- **Unit cost and warehouse are intentionally NOT captured at creation.** They are decided when the actual delivery is recorded. `quantity_requested` = sum of all line quantities. `total_amount` starts at 0.
- **DR number logic:** the DR number starts as the typed RIS number. If that DR already exists, a suffix is appended (`-1`, `-2`, ... up to 100 attempts) to keep the DR unique.
- All admins get a system notification: "DR #... has been created."

5.2 Recording the actual Delivery (the step that moves stock)

- For each subsidy line, the receiver enters: destination warehouse, unit cost, quantity delivered, and condition (default "good"). ENGAS unit cost may also be recorded.
- The system uses `Item::findOrCreateByUnitCost()` to locate the matching item: same warehouse + description + unit + category + unit cost (within ±0.001), and optionally same expiry. It never creates duplicate item records for the same cost—it reuses the existing one.
- If a matching item with a stock number exists, it is reused (and its ENGAS cost/account code synced if empty). If only a "draft" item (no stock number yet) exists, that draft is claimed, given a stock number, and updated with the cost. Otherwise a brand-new item is created with a fresh stock number.
- **Stock number format:** `<WAREHOUSE-CODE>-<first-3-letters-of-category>-<4-digit-number>`, e.g. `W1-FOO-0001`. The generator uses a database transaction with row locking so two simultaneous deliveries never get the same number.
- An item's `quantity` increases, a stock card **receipt** entry is written, and the item records its source subsidy (RIS + DR snapshot, status "active").
- The subsidy's `qty_delivered` is updated; status auto-computes to `pending` / `partial` / (complete).

5.3 Edit vs Correct (admin-only)

- **Edit**: changes header and lines of a document that has no deliveries yet (lines that already have deliveries are flagged as locked and cannot be removed).
- **Correct**: fixes a document after deliveries exist. Corrections are logged in the subsidy audit log (action types: `update`, `unit_cost_cascade`, `ris_cascade`, `supplier_cascade`) and cascade safely to dependent delivery/stock-card records.

5.4 Archive / Restore (admin-only)

- Archiving sets `is_archived = true` — a "frozen" state that removes the subsidy from active operations WITHOUT deleting it.
- Items and transfers delivered by an archived subsidy keep their snapshot (`source_subsidy_status = 'archived'`), so the trail survives.
- Restore flips it back.

5.5 Update Delivery (admin-only)

- Individual delivery records (a specific receipt event) can be edited by the full admin. Changes are validated and applied within a transaction, keeping stock quantities and stock cards consistent.

> 🟥 **Watch during QA:** Editing a delivery that was partially issued onward to a requisition or transfer can change downstream balances. The system performs cascades with audit logging, but QA should verify the resulting stock card math on every edit test.

6. RIS / Requisition Logic (in detail)

6.1 Lifecycle of an RIS

`pending` → (`approved` or `partially_approved`) → optionally later corrections. There is also a `cancelled` status.

6.2 Creating a Requisition

- Open to all authenticated users with a warehouse.
- Header: entity name, fund cluster, office, division, responsibility center code, requesting LGU, purpose, date requested, signatory names/designations (requested by, approved by, issued by, received by).
- Lines are **description-level**: the exact stock record is chosen later by the dispatcher. Each line: description, unit, quantity requested, optional expiry.
- `ris_number` must be unique.

6.3 Approval & Dispatch (the step that issues stock)

- Approvers: admin, warehouse manager, warehouse head, supply custodian (all roles except plain staff).

- On the approve screen the approver dispatches each line: choose the warehouse, the specific stock item, quantity issued, unit cost, ENGAS unit cost, DR number (optional), expiry. The form is fed by an AJAX endpoint that lists items available in the chosen warehouse.
- Each dispatch becomes a row in `requisition_dispatch_items` (each dispatch is its own record — so one RIS line can be dispatched in parts, possibly from different warehouses).
- Each dispatch decrements the source item, writes a stock card **issuance** entry linked by `dispatch_item_id` (so editing one dispatch touches exactly that entry), and records the issued DR/expiry.
- Status becomes `approved` when everything is fully issued, or `partially_approved` when only part is issued.

6.4 Editing a Dispatch (admin-only)

- Admin can edit an individual dispatch: change stock item, quantity, cost, DR, expiry. The system reverses the old stock-card effect and applies the new one inside a transaction, and logs the change in the requisition audit log.
- **Rules protecting integrity:** a requested quantity cannot be reduced below the already-issued quantity; lines already dispatched are locked in some respects.

6.5 Corrections

- Admin uses the correction screen for an approved RIS. Corrections write `requisition_audit_logs` rows with `{ field: { old, new } }` for both header and line changes.

6.6 Printing

- The print view renders the official RIS form with signatories.

7. Stock Transfer Logic (in detail)

7.1 Create Transfer (admin + warehouse manager)

- Fields: source warehouse, destination warehouse, transfer date, items with quantities, remarks.
- An AJAX endpoint (`/api/transfer-items`) lists items available in the chosen source warehouse.
- Center staff are blocked from creating transfers (403).

7.2 Dispatch Transfer (the step that moves stock)

- For each line: source item quantity decreases; the destination item is found or created using the same matching rules as delivery; destination quantity increases; stock card entries are written for both sides.
- The transfer keeps a **subsidy snapshot**: `delivery_subsidy_id` (nullable FK), `source_ris_number`, `source_dr_number`, `source_subsidy_status` (null/active, `deleted`, or `archived`). This lets a transfer be traced back to the original subsidy even after that subsidy is deleted.

7.3 Visibility rules

- A non-admin sees a transfer only if **either** the source or the destination warehouse is one of their assigned warehouses.
- Search (`q`) matches transfer number, source RIS number, or source DR number.
- Filter by source subsidy status (active / deleted / archived) is supported on the list.

7.4 Edit / Delete (admin-only)

- Editing and deleting a transfer is admin-only. Deleting a dispatched transfer reverses both stock movements and the stock card entries; the reversal history is kept in the transfer audit log with the transfer number snapshotted (because the transfer row itself is gone).

8. Stock Card Logic (in detail)

8.1 What a stock card entry is

Every stock card entry represents one movement of one item: a **receipt** (from delivery or transfer-in) or an **issuance** (from a requisition dispatch or transfer-out). Each entry stores:

- entry date and reference (DR number, RIS number, or transfer number) plus `reference_type` (delivery / issuance / adjustment)
- receipt qty / unit cost / total cost
- issue qty
- running balance qty / unit cost / total cost after the entry
- optional "days to consume" and from/to columns
- an optional link to the exact dispatch item that produced it (`dispatch_item_id`)

8.2 How the balance is computed

- Entries are stored chronologically (entry date, then id). Each new movement sets the new balance = previous balance ± movement.
- After a receipt, the balance unit cost becomes the receipt unit cost (the item's current cost); after an issuance, the balance total cost decreases by the issued quantity × unit cost.
- Because items are matched by unit cost, an item record effectively represents one cost variant; the FIFO (History by Unit Cost) view re-layers receipts and applies issues to the oldest batches first, so QA can verify cost valuation line-by-line.

8.3 ENGAS on the web pages (confirmed)

Screen	ENGAS shown?
Stock Cards index (`/stock-cards/{category}`)	☑ Yes — Engas Unit Cost and Engas Value shown when `engas_unit_cost` is set.
Stock Cards Summary (`/stock-cards/summary`)	☑ Yes — same as above.
Item History (`/stock-cards/item/{id}/history`)	☑ Yes — "ENGAS Value" stat card, shown only to admins when ENGAS cost is set.
History by Unit Cost	✗ No.
Print Stock Card (`/stock-cards/item/{id}/print`)	✗ No — intentionally uses standard unit cost only (this was the change that was later reverted to keep the printed form standard).
Transfers print / Requisitions print / RPCI print / RSMI print	✗ No.
RPCI and RSMI **Excel exports**	☑ Yes — an "Engas Unit Cost" column is present in both .xlsx exports, and Inventory Balance export too.

8.4 Access control

- Item history, FIFO view, and print are available to users who can access the item's warehouse (admin always; others only their assigned warehouses, else 403).
- The summary and category views are warehouse-scoped for non-admins.

9. Inventory Balance & Valuation

9.1 What it reports

- **Admin-only** screen (`/reports/inventory-balance`); non-admins get 403.
- Shows every active item with quantity > 0, grouped: warehouse → category → item name → individual inventory records.
- Columns: warehouse, account code, category label, description, unit, quantity, **total quantity** (sum of all records sharing the same item name + warehouse, ignoring unit cost / ENGAS / expiry), unit cost, ENGAS unit cost, total value (quantity × unit cost).
- Category and warehouse grand totals in pesos are computed.
- Filters: warehouse, category, item, and source subsidy status.

9.2 Valuation rules

- Unit value is the item's stored `unit_cost` (the cost at the time of its last receipt).
- ENGAS unit cost is kept separately and never affects the standard valuation/totals — it is an informational column.
- Because the same item name can exist as multiple cost variants, **every record keeps its own row**; only the "Total Qty" is merged once per item name + warehouse group.
- Grand totals are sums of quantity × unit_cost across all rows.

9.3 Exports

- All three reports (RPCI, RSMI, Inventory Balance) can be downloaded as .xlsx files built with PhpSpreadsheet, with the same title rows, category headers, borders, number formats, and grand totals as the on-screen report.

10. Database Explanation

The database name is `wgims` (MySQL). Below is every table and what it stores, written for non-programmers. The names in parentheses are the real database table names.

10.1 People & Organizations

Table	Stores

`users`	Login accounts: username, name, email, password (hashed), role, primary warehouse, active flag.
`warehouses`	Warehouses/centers: name, code (e.g. W1), place, active flag.
`user_warehouse`	Link table: which warehouses each non-admin user is assigned to (many-to-many).
`suppliers`	Suppliers: name, address, TIN, contact person, phone, email, active flag.

10.2 Catalog & Items

Table	Stores
`item_categories`	Categories (FOOD, NON-FOOD): key, label, account code, active flag, sort order. Pre-seeded with the two standard categories.
`item_catalog_items`	Master list of item names under each category (each with its own account code) used as the dropdown source in the delivery form.
`items`	The actual stock records: stock number, description, RIS no., unit, category, account code, warehouse, unit cost, quantity, quantity per item, reorder point, expiry, active flag, ENGAS unit cost, and subsidy-source snapshot (subsidy ID + RIS + DR + status).

10.3 Receipt Documents

Table	Stores
`delivery_subsidies`	The delivery/subsidy document: DR number (unique), supplier, warehouse (nullable until recorded), creator, date, RIS no., place of delivery, dates, total amount, quantity requested, status, remarks, archived flag.
`delivery_subsidy_items`	Lines of the document: which catalog item, description, unit, category, account code, warehouse, quantity, unit cost, amount, quantity delivered so far, expiry.
`deliveries`	Each actual receipt event against a subsidy: DR no., received-by user, delivery date, batch no., condition, total quantity delivered, remarks.

`delivery_items`	Each line of a receipt: the subsidy line it fulfills, the item it added stock to, warehouse, unit cost, quantity delivered, condition, DR no., ENGAS cost.

10.4 Issuance Documents

Table	Stores
`requisitions`	The RIS document: unique RIS no., DR no., warehouse, creator, approver, entity, fund cluster, office, division, responsibility center code, requesting LGU, purpose, dates, status, and signatory names/designations.
`requisition_items`	Lines of the RIS: description + unit (snapshots), optional linked item, warehouse, quantity requested, quantity issued, stock-available flag, unit cost, expiry, remarks.
`requisition_dispatch_items`	Each actual issuance (dispatch): the RIS line it belongs to, the exact stock item issued, quantity, unit cost, ENGAS cost, expiry, DR no., who created it.

10.5 Movement & Ledger

Table	Stores
`stock_transfers`	Transfer document: unique transfer no., source and destination warehouses, date, who did it, status, remarks, plus subsidy snapshot (source RIS/DR/status).
`stock_transfer_items`	Lines of a transfer: the source item, the destination item (created on dispatch), quantity requested, quantity, unit cost.
`stock_card_entries`	The running ledger: every receipt/issuance with date, reference + type, receipt qty/cost, issue qty, and the running balance (qty, unit cost, total cost). Optional link to the dispatch that caused an issuance.

10.6 Reports & Audit

Table	Stores

`report_snapshots`	Saved copies of RPCI/RSMI reports for a month period (JSON snapshot, serial no., who saved it).
`system_notifications`	Per-user notifications (title, message, type, link, read flag).
`delivery_subsidy_audit_logs`	History of subsidy updates/corrections with changed fields and cascade summary.
`requisition_audit_logs`	History of RIS corrections (changed fields, old/new values).
`stock_transfer_audit_logs`	History of transfer create/update/dispatch/deletion; keeps the transfer number even after the transfer is deleted.

10.7 Relationships (who points to whom)

- Item → Warehouse (one item is stored in one warehouse).
- Delivery/Subsidy → Supplier, Warehouse, Creator; → Delivery Subsidy Items; → Deliveries; → Delivery Items; → Items.
- Requisition → Warehouse, Creator, Approver; → Requisition Items; → Requisition Dispatch Items; → Items; → Stock Card Entries (via dispatch).
- Stock Transfer → Source/Destination Warehouses; → Stock Transfer Items; → Source and Destination Items; → Stock Card Entries.
- Stock Card Entry → Item (many entries per item).
- Users ↔ Warehouses (many-to-many via `user_warehouse`).

> 🌑 **Migrations naming quirk (from the source code, not an error in the app):** the migration files are named after what the developer originally planned, not always what they create. Two examples: `2026_05_05_100003_create_purchase_orders_table.php` actually creates `delivery_subsidies` and `delivery_subsidy_items`; and `2026_05_05_100007_create_notifications_table.php` actually creates `system_notifications`. The tables are correct — only the file names are misleading. This is harmless at runtime but worth knowing if an evaluator reads the migration folder.

11. Routes (URL Map)

All pages require login except the login page itself. The map below uses `{id}` for a record id. Middleware notes explain who may open each URL.

URL	Purpose	Access

`/login` (GET/POST), `/logout` (POST)	Login and logout	Public / any user
`/`	Dashboard	Any logged-in
`/items`, `/items/{id}`	Item list and detail	Any logged-in (scoped)
`/items/create`, POST `/items`	Add item	Admin + WM
`/items/{id}/edit`, PUT/DELETE `/items/{id}`	Edit / delete item	Admin only
`/item-categories` and sub-routes	Manage categories + catalog item names	Admin only
`/delivery-subsidies`	List	Any logged-in (scoped)
`/delivery-subsidies/create`, POST `/delivery-subsidies`	Create subsidy	Admin + WM
`/delivery-subsidies/{id}`	Detail	Any logged-in (scoped)
`/delivery-subsidies/{id}/delivery` (GET/POST)	Record delivery	Admin + WM
`/delivery-subsidies/{id}/edit`, `/edit-data`, `/correction-data`, PUT `/correct`, PUT `/{id}`, DELETE `/{id}`	Edit / correct / delete	Admin only
`/delivery-subsidies/{id}/archive`, `/restore`	Archive / restore	Admin only
`/delivery-subsidies/{id}/deliveries/{did}/edit`, PUT `/{did}`	Edit a delivery event	Admin only
`/delivery-subsidies/{id}/audit-log`	Subsidy audit log	Admin only

`/requisitions`, `/requisitions/create`, POST `/requisitions`, `/requisitions/{id}`	RIS list / create / detail	Any logged-in (scoped)
`/requisitions/{id}/approve` (GET/POST)	Approve + dispatch	Approver roles (not staff)
`/requisitions/dispatch/{did}/edit-data`, PUT `/requisitions/dispatch/{did}`	Edit a dispatch	Admin only
`/requisitions/{id}/edit`, PUT, `/correct`, `/audit-log`	Edit / correct / log	Admin only
DELETE `/requisitions/{id}`	Delete RIS	Admin only
`/requisitions/{id}/signatories` (GET/PUT), `/print`	Signatories & print	Any logged-in (scoped)
`/stock-cards`, `/stock-cards/summary`, `/stock-cards/{category}`	Stock card summaries	Any logged-in (scoped)
`/stock-cards/item/{id}/history`, `/history-by-cost`, `/print`	Item stock card views	Warehouse-scoped
`/transfers`, `/transfers/{id}`	Transfer list / detail	Any logged-in (scoped)
`/transfers/create`, POST `/transfers`, POST `/transfers/{id}/dispatch`	Create / dispatch transfer	Admin + WM
`/transfers/{id}/edit`, PUT, DELETE	Edit / delete transfer	Admin only
`/transfers/{id}/print`	Print transfer	Any logged-in (scoped)
`/suppliers`, `/suppliers/create`, POST	Supplier list / create	Create = Admin + WM
`/suppliers/{id}/edit`, PUT, `/toggle`	Edit / toggle supplier	Admin only

`/reports/rpci`, `/print`, `/export`, POST `/snapshot`	RPCI report	Any logged-in (scoped)
`/reports/rsmi`, `/print`, `/export`, POST `/snapshot`	RSMI report	Any logged-in (scoped)
`/reports/inventory-balance`, `/export`	Inventory balance	Admin only
`/reports/snapshot/{id}`	View saved report snapshot	Scoped to warehouse
`/warehouses`, `/warehouses/create`, POST, `/warehouses/{id}/edit`, PUT	Warehouse management	Create = Admin + WM; edit = Admin
`/users`, `/users/create`, POST, `/users/{id}/edit`, PUT	User management	Admin only
`/notifications`, `/read`, `/read-ajax`, `/read-all`, `/api/notifications/unread`	Notifications	Own notifications only
API helpers: `/api/check-dr`, `/api/item-stock-card`, `/api/requisition-items`, `/api/requisition-description-items`, `/api/transfer-items`, `/api/check-username`	AJAX data for forms	Logged-in

12. Roles & Permissions

The system does **not** use a permission table — access is controlled by the `role` field on each user plus middleware. (The Spatie permission tables that were created once were never used and were removed from the database; the code uses its own hard-coded role logic.)

12.1 The five roles

Role (code)	Display label	Meaning
`admin`	Administrator	Full access to everything, every warehouse. Can write and delete.

`warehouse_manager`	Warehouse Manager	Sees everything like admin, can **create** documents, can approve; but **cannot edit/delete**.
`supply_custodian`	Supply Custodian	Warehouse-scoped; can approve and manage their warehouse.
`center_head`	Warehouse Head	Warehouse-scoped; can approve.
`center_staff`	Warehouse Staff	Warehouse-scoped, view + record deliveries; cannot create documents or approve. (This is the default role for a new user.)

12.2 Capability matrix (what each role can do)

Action	Admin	Warehouse Manager	Supply Custodian	Warehouse Head	Warehouse Staff
View all warehouses' data	☑	☑	✗ (own only)	✗ (own only)	✗ (own only)
View own warehouse data	☑	☑	☑	☑	☑
Create Delivery/Subsidy, Item, Supplier, Warehouse, Transfer	☑	☑	✗	✗	✗
Record a delivery (receive stock)	☑	☑	⚠ via create-capable flows only	⚠	✗ (cannot create documents)
Approve a requisition (dispatch stock)	☑	☑	☑	☑	✗
Edit / delete items, subsidies, transfers, suppliers	☑	✗	✗	✗	✗
Manage users / categories	☑	✗	✗	✗	✗
View Inventory Balance report	☑	☑ (admin access)	✗	✗	✗

See ENGAS value on stock card pages	☑	☑ (admin access)	Only where shown to all	Only where shown to all	Only where shown to all
Edit ENGAS unit cost on an item	☑	✗	✗	✗	✗

Middleware used (aliases defined in `bootstrap/app.php`): `admin` = admin + WM; `admin.write` = admin only; `admin.create` = admin + WM; `admin.only.strict` = admin only (used for users and categories).

> **Warehouse scoping (important for QA):** a non-admin user sees *only* records tied to their assigned warehouses (checked via both the legacy `warehouse_id` column and the `user_warehouse` assignment table). If a user has no warehouse at all, they see an empty list. For transfers, the user can see a transfer if *either* end (source or destination) is theirs.

13. Validation & Business Rules

13.1 User / Account

- Username: required, max 50, unique. Name: required, max 100. Email: optional, must be valid, unique. Role: must be one of the five roles. Password: required, min 8 characters, must match confirmation.
- Non-admin/non-manager roles MUST be assigned at least one warehouse.

13.2 Item

- Description required (max 255). Unit required and must be from the system's unit list. Category required and must be a configured category.
- Quantity per item: positive integer if provided. Reorder point: 0 or more. Expiry: valid date. ENGAS unit cost: 0 or more (admin-only field).
- Non-admins can only add items to one of their assigned warehouses.

13.3 Delivery/Subsidy

- RIS number required; supplier required and must exist; date required.
- At least one line; each line: description required, unit required, category must be valid, quantity ≥ 0.01.
- DR number must be unique (auto-suffixed on collision).
- Lines that already have deliveries cannot be removed via edit.

13.4 Requisition / RIS

- RIS number unique. Purpose required. Date requested required.
- Approval requires choosing a real stock item for each issued line; issued quantity cannot exceed the available quantity in the source warehouse.
- Correction rules: you cannot reduce a requested quantity below what has already been issued; dispatched lines are protected.

13.5 Transfer

- Source and destination warehouses required and must differ; transfer date required; at least one line with a quantity; quantity cannot exceed the source warehouse's available quantity.

13.6 General business rules

- Stock is never edited directly — it changes only through documents (delivery, dispatch, transfer).
- Item records are de-duplicated by (warehouse, description, unit, category, unit cost ± 0.001, optionally expiry).
- Every deletion that affects stock is either blocked (item in use) or reversed with an audit trail (transfer/sub-subsidy cascade).

14. Delete Behavior

Deletion is deliberately conservative because inventory is financial. Here is exactly what happens when each kind of record is deleted.

14.1 Item deletion

- **Blocked with a clear message** if the item is referenced by any delivery/subsidy line, any requisition line, or any requisition dispatch. The system tells you which kind of record is blocking it.
- Otherwise: its stock card entries are deleted first, then the item itself. (Only reachable if the item truly has no document history — in practice most real items are protected.)

14.2 Delivery/Subsidy deletion

- Admin-only. Deleting a subsidy that has deliveries triggers a cascade service that reverses the stock effects (decrease quantities, remove stock card entries), and the dependent

items/transfers keep the subsidy's RIS/DR snapshot with status `deleted` so the "FROM DELETED SUBSIDY" trail remains.
- An audit entry is written *before* the subsidy row is removed, and the audit log survives the delete (its foreign key is set to null-on-delete).
- Archiving is offered as a safer alternative (freeze without deleting).

14.3 Requisition deletion

- Admin-only. Deleting a requisition cascades to its line items; if stock was already dispatched, the dispatch effects (stock decreases, stock card entries) are reversed so quantities stay correct.

14.4 Transfer deletion

- Admin-only. Deleting a dispatched transfer reverses both stock movements and stock card entries; the audit log keeps the transfer number even though the transfer row is gone.

14.5 Category / catalog item deletion

- A category cannot be deleted while items are assigned to it — you must deactivate it instead.
- A catalog item name cannot be deleted if any delivery/subsidy line uses it — deactivate it instead.

> 🟥 **QA tip:** always test "delete" as an evaluator. Expect either a friendly error explaining why a delete is blocked, or a successful delete that leaves the stock quantities and stock cards mathematically consistent. Never expect a silent partial delete.

15. Audit Logging

The system logs who changed what, and keeps the trail even after records are deleted. There are three audit tables (all admin-viewable):

Log	Records	Survives delete?
Subsidy audit log	Updates and corrections with changed fields (old/new) and a summary of cascaded records; actions: `update`, `unit_cost_cascade`, `ris_cascade`, `supplier_cascade`.	☑ Yes (FK set to null-on-delete, entry written before deletion)

Requisition audit log	Corrections with `{ field: { old, new } }` for header and line changes; action defaults to `correction`.	❌ No (deleted with the requisition)
Transfer audit log	Create / update / dispatch / `reversed_deleted` events with changed fields and the transfer number snapshotted.	☑ Yes (transfer number stored on the log row)

Notifications are also part of the trail: admins are notified when a delivery/subsidy is created, and each user has their own notification inbox (read/unread).

> ☑ **QA-friendly behavior:** the correction and cascade paths log before/after values in JSON. When an evaluator asks "can you prove who changed this DR's unit cost?" the answer is: open the Subsidy Audit Log.

16. Error Handling

- **Validation errors** are shown on the form with a message under the offending field (standard Laravel behavior). Examples: "The password must be at least 8 characters", "The quantity must be at least 0.01".
- **Authorization errors** return HTTP 403 with a plain-language message, e.g. "Access denied. Admin only.", "Access denied. You have read-only access.", "You can only add items to a warehouse assigned to you."
- **Not logged in** → you are redirected to the login page (except on API/JSON calls, which return 403 JSON or redirect as appropriate).
- **Not found** → standard 404 page.
- **Unexpected server errors** → Laravel error page. Because `APP_DEBUG=true`, the page shows a stack trace during development. Set `APP_DEBUG=false` in production for a generic error page.
- **Business-rule blockers** (e.g., deleting an item that is in use) do not throw errors — they redirect back with a clear flash message: "Cannot delete ... it is referenced by one or more Deliveries / Subsidies."
- **Login rate limiting**: the login attempt is rate-limited in the controller (too many attempts from the same client are throttled), protecting against brute force.

> 🌑 **QA note:** because debug mode is on, a deliberate error test will show a technical error page. This is normal in a dev environment. In production the same error would show a simple "500" page. Mention this to evaluators so they don't mistake it for a defect.

17. Performance

Several deliberate performance decisions are visible in the code:

- **DB-level aggregation on the dashboard** — the admin dashboard computes account balances with a single SQL `SUM(...) GROUP BY` query instead of loading every item into memory.
- **Indexes on hot columns** — dedicated migration(s) add indexes for performance (e.g., warehouse/status combos, item lookups by description/stock number, audit-log (record id + created_at) lookups).
- **Bulk inserts** — notifications to all admins are inserted in one batch query instead of one query per admin.
- **Targeted queries** — warehouse-scoping uses `whereIn` and `exists` checks rather than pulling full collections; e.g., `hasWarehouse()` runs an `exists()` query.
- **Paginated lists** — list pages (items, subsidies, requisitions, users, suppliers, warehouses, notifications) paginate at 20 rows.
- **De-duplication on delivery** — delivering stock reuses existing item records instead of creating duplicates, keeping the item table small and balances grouped.

> 💡 **Recommendation for QA:** test with a large dataset (e.g., hundreds of items and documents). The design is built to stay responsive, but a QA load test with real-scale data is the best proof.

18. Security

- **Passwords hashed** — stored with bcrypt/Argon-style hashing, never plain text.
- **Server-side authorization everywhere** — roles and warehouse scoping are enforced in the controllers and middleware, not just hidden in the UI. Directly typing an admin URL as a normal user returns 403.
- **Ownership checks** — notifications can only be marked read by their owner; a foreign notification returns 403.
- **CSRF protection** — all POST/PUT/PATCH/DELETE forms include Laravel's CSRF token.
- **SQL injection protection** — all queries use Laravel's query builder / Eloquent (parameter binding).
- **Login throttling** — rate-limited login attempts.
- **Session-based auth** — standard Laravel sessions; logout clears the session.
- **Mass-assignment protection** — controllers use validation + explicit `fillable` fields; e.g., suppliers are created from the validated array, not raw request data.
- **ENGAS admin-gating** — ENGAS unit cost can only be set/edited by the full admin role.

> 🛑 **Known areas to check during security QA:**
>
> 1. `APP_DEBUG=true` is a dev-only setting (see section 16).
> 2. Confirm HTTPS/SSL is applied in the production server (not visible in code).
> 3. Password strength policy is minimum 8 characters (a stronger policy can be added).
> 4. The login form is not a two-factor flow (standard single password login).

19. Migrations (Database History) Review

Migrations are the versioned scripts that create the database. There are 42 migration files. Notable findings for evaluators:

Migration (date)	What it actually does
`0001_01_01_000000` … `000002`	Laravel base tables: users, password resets, sessions, cache, jobs.
`2026_05_05_100000` – `100008`	Core schema: warehouses, suppliers, items, delivery_subsidies (+ items), deliveries (+ items), requisitions (+ items), stock_card_entries, system_notifications, report_snapshots.
`2026_05_14_000001`	Stock transfers (+ items).
`2026_05_18_000001`	User-warehouse pivot table, backfilled from existing users.
`2026_05_20_000001`	Performance indexes.
`2026_06_10_000001`	Delivery/subsidy audit log.
`2026_06_13_000001`	Item categories (+ seed of FOOD and NON-FOOD).
`2026_06_14_000001`	Optimized database indexes.
`2026_06_21_102551`	Spatie permission tables (roles/permissions) — **later removed** (never used; the app uses its own role system).
`2026_06_22_000001`	Adds `engas_unit_cost` to items.
`2026_07_01_000001`	Adds `warehouse_id` to delivery_subsidy_items.
`2026_08_10_000001` – `000002`	Makes subsidy warehouse fields nullable (multi-warehouse support); creates item_catalog_items + catalog/account-code links on subsidy lines.
`2026_08_11_000001` – `000080`	Deliveries/items evolution: DR number on delivery items, expiry on subsidy lines, ENGAS costs on delivery items, warehouse on delivery items; requisitions become description-level (nullable item_id + snapshots), dispatch fields, dispatch items table, requesting LGU, nullable DR.

`2026_08_12_000001` – `000005`	Dispatch link on stock card entries; requisition + transfer audit logs; subsidy link + snapshots on transfers; archive flag + transfer DR snapshot.
`2026_08_13_000001` – `000002`	Item subsidy-source snapshot (backfilled); transfer destination lineage backfill.
`2026_08_13_100001`	Drops the unused Spatie permission tables (keeps the file for history).

> 💡 **The migration history is clean and non-destructive:** each step adds or adjusts columns, and backfill steps preserve existing data. The only removed tables were the never-used permission tables.

20. QA Q&A (Frequently Asked Questions)

These are the questions evaluators are most likely to ask, with the correct answers based on the code.

Question	Answer
How do I know a delivery really increased stock?	Record a delivery with a unit cost, then open the item's Stock Card History and the Items list. You will see a receipt entry and a higher quantity, and the Inventory Balance / RPCI will reflect it.
Why can't I delete this item?	Because it is already referenced by a delivery/subsidy or a requisition. The system intentionally blocks deletion to protect the audit trail; deactivate or archive instead.
What does "partial" mean on a subsidy?	Some lines (or some quantity) have been delivered, but not all. The document is still open for further deliveries.
What does "partially_approved" mean on an RIS?	The approver issued only part of the requested quantity (or only some lines). The remainder was not dispatched.
What is the difference between Edit and Correct on a subsidy?	Edit changes the document before deliveries exist. Correct fixes it after deliveries exist and logs the change with a cascade summary.
What is a stock number and where does it come from?	A unique per-warehouse-per-category number like W1-FOO-0001, auto-generated at first delivery. It is never typed by the user.

What is ENGAS unit cost?	A separate informational cost kept per item (used for valuation comparisons). It does not change the standard unit cost or totals, and only the admin can edit it.
Can two users work at the same time?	Yes. The system runs on a normal web server with per-user sessions; concurrent work is expected. Stock number generation is locked to avoid duplicates.
What happens to stock when a subsidy that delivered items is deleted?	The stock effects are reversed, and the items keep a "FROM DELETED SUBSIDY" trail (RIS/DR snapshot) so the history remains explainable.
Is there a way to see who changed what?	Yes. Audit logs exist for subsidies, requisitions, and transfers (admin-only screens), storing before/after values.
Why does the Warehouse Manager see everything but cannot edit?	By design: the WM role has admin-level visibility and can create/approve, but write/edit/delete is reserved for the full admin.
How are reports saved for a past month?	Use the snapshot feature on the RPCI/RSMI report pages. It stores the exact numbers at that moment, and you can re-open the snapshot later.
What if an item is out of stock?	It stays in the system marked inactive or at zero quantity; the RPCI still lists it (per physical count conventions). Admins see it; filters can isolate out-of-stock and low-stock items.

21. QA Test Scenarios

These scenarios mirror what a government QA/evaluator typically tests. Each row gives the test, the expected result, and which role to use.

21.1 Authentication & Roles

Scenario	Expected result
Login with wrong password	Error message; after several tries, throttled.
Login as each of the 5 roles	Each sees a dashboard scoped to their role; menus reflect permissions.

Staff tries to open `/delivery-subsidies/create` directly	403 "Access denied."
Warehouse Manager tries to open `/items/{id}/edit`	403 "read-only access".
Non-admin tries `/users` or `/item-categories`	403 (admin-only).
User with no warehouse assigned	Dashboard shows "no warehouse" notice; lists are empty, not errors.

21.2 Receipt (Delivery/Subsidy)

Scenario	Expected result
Create subsidy with 2 lines, then record delivery of line 1 only	Subsidy status = partial; item quantities increase for line 1 only; stock cards show receipts.
Record full delivery of all lines	Subsidy fully delivered; quantities and stock cards updated; RPCI shows the new stock.
Create subsidy with duplicate DR (same RIS number)	Auto-suffix keeps DR unique (`-1`).
Try to create with zero/empty line	Validation error; nothing saved.
Deliver same item twice at different unit costs	Two separate cost variants (items) are created; no duplication of the original cost variant.
Archive a subsidy with delivered stock	Subsidy removed from active ops; items keep "archived" trail; restore brings it back.

21.3 Issuance (Requisition)

Scenario	Expected result

Create RIS requesting more than available	Approve screen allows issuing only what is available; status becomes partially_approved; stock never goes negative.
Approve and fully dispatch an RIS	Stock decreases, stock card issuance entries appear, status = approved.
Edit a dispatch after approval	Only the exact stock card entry is reversed/updated; quantities stay consistent.
Attempt to reduce requested quantity below issued	Blocked by the correction rule with an error.

21.4 Transfers

Scenario	Expected result
Transfer 50 pcs from W1 to W2 and dispatch	W1 item -50, W2 item +50 (or created), stock cards on both sides, transfer shows source subsidy trail.
Transfer more than available	Blocked; no negative stock.
Staff opens create transfer directly	403.
Delete a dispatched transfer	Both sides reversed; audit log keeps transfer number.
Non-admin of W1 views a W1→W2 transfer	Visible (either end is theirs).

21.5 Stock Cards, Reports & Valuations

Scenario	Expected result
Check running balance after receipt then issuance	Balance = previous ± movement, consistent with the item quantity.
Check FIFO (history by unit cost) with two cost batches	Issues are consumed from the oldest batch first; remaining batch quantities match.

Print RPCI and RSMI	Official report format with headers, categories, account codes, subtotals/grand totals.
Export RPCI / RSMI / Inventory Balance to Excel	.xlsx downloads with the same columns, including Engas Unit Cost where applicable.
Save an RPCI snapshot for a month, then re-open	Snapshot shows exactly the numbers at save time, even if data changed since.
Non-admin opens Inventory Balance	403 (admin-only).

21.6 Data Integrity & Edge Cases

Scenario	Expected result
Two users deliver to the same warehouse/category at the same moment	Unique stock numbers (locked generation) — no duplicates.
Delete an item with documents	Blocked with an explanatory message.
Correction on a subsidy cascades to deliveries	Audit log shows the cascade summary; downstream numbers updated consistently.
Open a notification of another user	403.
Search/filter combinations on items and transfers	Correct filtered results with pagination and query strings preserved.

22. End-to-End Example (with real numbers)

Follow one box of rice through the whole system to see how every module connects.

1. **Setup:** Warehouse W1 (code `W1`) exists. Supplier "MegaMart" exists. Admin creates user "Ana" (custodian, warehouse W1).
2. **Receive:** Ana records a Delivery/Subsidy: DR/RIS `RIS-2026-001`, supplier MegaMart, line "Rice 25kg", unit "sack", category FOOD, quantity 100.

3. **Record delivery:** Ana records the actual arrival at W1: unit cost ₱1,200.00/sack, 100 sacks delivered, condition good. The system creates item `W1-FOO-0001` — Rice 25kg — qty 100, unit cost 1,200.00. Stock card gets one receipt entry: balance 100. Subsidy qty_delivered = 100. RPCI now shows 100 sacks of Rice at W1 worth ₱120,000.00.
4. **Request:** Staff of W1 creates RIS `RIS-2026-002` requesting 40 sacks of Rice 25kg.
5. **Approve/dispatch:** The approver dispatches 40 sacks from `W1-FOO-0001` (DR no. noted, cost 1,200.00). Item qty becomes 60. Stock card issuance entry: balance 60. RIS status = approved. RSMI now reports 40 sacks issued.
6. **Transfer:** W1 sends 20 sacks to W2 (a new warehouse). Create transfer, dispatch. W1 item qty 60→40. W2 gets its own item (e.g., `W2-FOO-0001`, qty 20, cost 1,200.00). Stock cards record both sides. The transfer shows source subsidy RIS-2026-001/DR for tracing.
7. **Verify:** Inventory Balance (admin) shows W1 Rice 40 sacks, W2 Rice 20 sacks, grand total 60 sacks = ₱72,000.00. Item stock card history shows: +100 (delivery), -40 (RIS-2026-002), -20 (transfer out). Everything reconciles.

23. WGIMS Explained in 10 Minutes

Read this aloud to an evaluator who is new to the system.

"WGIMS is the electronic stock room ledger of the welfare office. Every item has a home warehouse, a stock number, a quantity, and a cost. Nothing enters or leaves the ledger by hand — it only changes through three kinds of official documents: a Delivery/Subsidy brings stock in, a Requisition (RIS) takes stock out to a requesting unit, and a Stock Transfer moves stock between warehouses.

When goods arrive, someone creates a Delivery/Subsidy document and records the actual receipt. The system gives each item a unique stock number, increases the quantity, and writes an entry in the item's stock card — which is the running history, like a bank statement. When goods are requested, an approver issues them against the actual stock; the system decreases the quantity and writes the issuance on the same stock card. Transfers do both at once: out of one warehouse and into another, with the history preserved on both sides.

Because every movement is a document, every report is simply a view of the ledger: RPCI counts what is physically on hand, RSMI lists what was issued, and the Inventory Balance summarizes everything by warehouse and category. Reports can be printed in the official government format or exported to Excel.

Security is role-based: an Administrator can do everything; a Warehouse Manager can see everything and create documents but cannot edit or delete; Supply Custodians, Heads, and Staff manage only their own warehouse, and only Custodians and Heads can approve requests. Users are locked to their assigned warehouses, so no one sees another center's stock. The system also keeps audit logs of every correction, and it protects the books by refusing to delete anything that is already part of history — you archive it or correct it instead.

In short: WGIMS makes the physical count always match the books, and it produces the official reports the office needs, with a full trail of who did what."

24. Symbol Legend & Honesty Notes

Symbols used throughout this guide (and by convention in the codebase):

Symbol	Meaning
☑	Implemented and verified in the source code.
⚠	Implemented with limitations / partial coverage.
✕	Not implemented (by design or absent).
🔘	Potential problem / something to flag during QA or before production.
💡	Recommended improvement or next step.

Do not guess: Every statement in this guide was checked against the application source code (controllers, models, routes, migrations, middleware, views). Where something could not be confirmed from code alone (e.g., production server SSL, real user data, live network settings), it is marked as such rather than assumed.

25. Final Deliverable Notes

- **Document purpose:** this guide is the single reference to explain WGIMS to QA testers and evaluators.
- **How to run the system for a demo:** start XAMPP (Apache + MySQL), then serve the app (e.g., `php artisan serve`) and open the app URL in a browser. Log in with an existing user or create one.
- **How to run the automated tests:** from the project folder, run `php artisan test`. The suite runs against an in-memory database and does not touch the real data. Current status: the feature suite passes; one pre-existing sample test (ExampleTest) fails because it hits the home page without logging in and gets redirected — this is expected and unrelated to the application's features.
- **Suggested demo script (5 minutes):** (1) log in as admin and show the dashboard; (2) show one delivery/subsidy and its stock card; (3) show one approved RIS; (4) run the RPCI report and export it; (5) open the audit log of a corrected record.
- **Suggested follow-ups before production:** set `APP_DEBUG=false`, enable HTTPS, back up the database, review user account policies, and run a data-scale load test.

— End of Document —